TECHNICAL WHITEPAPER · AGENT SECURITY

PANOPTES

Toward Survivable Security for LLM Agents

A forensic critique of OpenAPPA and its policy algebra — the deterministic guardrail that recorded zero attacks across 1,320 guarded evaluations — and a nine-organ architecture for the residue no algebra can close. Grounded in the NeurIPS 2026 workshop paper (arXiv:2607.24625), its benchmark tables, and four decades of information-flow theory.

A Position Paper on Trust Infrastructure for Agentic AI

Independent technical assessment · prepared from primary sources

OCTOBER 2026 · PREVIEW & RFC ERA

Contents

Executive Summary 1

1 The Landscape: Why Agent Security Broke in 2026 2

2 APPA: What It Is and What It Actually Proves 3

3 The Flaw Taxonomy: Where "Airtight" Leaks 6

4 The Five Walls: What Forty Years of Theory Actually Proves 8

5 PANOPTES: An Architecture for Survivable Agent Security 9

6 The Crown Jewel: Counterfactual Noninterference Adjudication 14

7 Roadmap: Contribute to APPA or Build PANOPTES? 16

8 The Residual Risk Budget: An Honest Closing Account 17

Engineering Annex — The Reference Implementation 18

References 20

Executive Summary

This paper examines **OpenAPPA**, the open-source deterministic guardrail for LLM agents published by Archestra and described in a NeurIPS 2026 workshop paper (arXiv:2607.24625v2), and argues two things at once. First, APPA is the most serious agent-security artifact currently shipping: its core idea — enforcing information-flow policy with a deterministic label algebra, then *recovering* blocked actions through sanitizers, scoped approvals, and subagent isolation instead of failing flat — is the correct architecture, and its measured results are strong. Second, its guarantees are narrower than its marketing: every load-bearing claim is conditional on label accuracy, mediated channels, and a trusted computing base whose endpoints are partly probabilistic. The paper itself says this plainly in its limitations section; the project page says "100% resistant to data exfiltration" instead. Both statements are true in their respective frames, and the gap between them is where this paper lives.

0 / 1,320	**64.2–91%**	**88.0 → 35.0**	**+4.22%**
attacks observed against guarded APPA across all evaluated episodes (paper)	utility range across 6,600 controlled episodes, three models (paper)	Bench-Corp task completion with recovery machinery progressively removed (ablation)	token overhead vs. stock agent on the Tau banking benchmark (evaluation page)

From that gap we derive a flaw taxonomy with five structural entries: probabilistic endpoints inside the trusted computing base; the semantic gap between world facts and finite labels; an enforcement boundary that covers mediated tool calls but not sandbox egress or covert channels; a monotone ratchet with no declassification lifecycle that measurably eats utility on memory-heavy tasks; and benchmarks that are partly self-authored. None of these are refutations — they are the known, inherited shape of forty years of information-flow control research, now transplanted onto agents, with genuinely novel recovery machinery bolted on.

The constructive answer is **PANOPTES**, a nine-organ architecture for survivable — not perfect — agent security. Its centerpiece is **counterfactual noninterference adjudication (CNI)**: defining leakage causally, in the sense of Pearl, as a property of the model's decision function rather than of the data, and testing it at runtime by replaying boundary actions with secrets swapped for inert dummies. CNI addresses the exact residue class — covert tool-choice channels, sanitizer-evading semantic leaks, split secrets — that no label algebra can see, because that residue lives in the decision function, not in the bytes. Around it: capability-based reach instead of named reach, injection channels with provable bandwidth caps, an immune system of synthesized canaries, gas-metered risk pricing with slashing, escrowed time for irreversible effects, n-version policy enforcement with proof-carrying action certificates, a policy constitution outside the agent's reach, and a research program to converge noninterference into the substrate itself.

The strategic recommendation is sequenced contribution rather than competition: feed provenance-channel labels and policy model-checking into APPA's open RFC while it is still soft; build the CNI prototype around APPA as its fast path; and let the killer result — an attack that passes the algebra and is caught by the adjudicator, reproduced on APPA's own harness — become both the paper and the product. "Airtight,"

redefined honestly, is not zero leaks. It is a known, attributable, reversible, unprofitable, and shrinking leak rate. The asymptote, not the wall.

1 The Landscape: Why Agent Security Broke in 2026

A coding agent in 2026 reads bug reports, browses the web, queries CRMs, writes to memory, opens pull requests, and sends email — often in one trajectory. Every one of those surfaces is simultaneously a work input and an attack surface. The canonical incident shape is quiet and multi-turn: an agent reads a bug report that contains a customer's email address; three turns later, filing a public GitHub issue, it pastes that address into the description. No single call looks wrong. The leak is a property of the *composition* of calls, not of any one call, and that is precisely what per-call judgment was never able to see.

The industry's dominant answer has been a second model watching the first. Claude Code's auto mode, OpenAI's Codex auto-review, and similar "auto-modes" route each risky tool call to a judge model before dispatch. Three structural problems follow. The judge is itself prompt-injectable, so harnesses hide tool outputs from it — meaning the judge rules on actions whose data it has not seen. It evaluates calls one at a time, so cross-call flows are invisible to it by construction. And it is probabilistic: OpenAI reportedly states that Codex auto-review catches 99.3% of prompt injections in its synthetic evals (as relayed by Bytes #525 and OpenAPPA's project page), which sounds decisive until you multiply the residual by millions of daily calls. A 0.7% miss rate at agent scale is not an edge case; it is a breach pipeline.

The deterministic counter-position — blocklists, regexes, allowed-tool lists — fails from the other side. Block *rm -rf /* and the model writes a Python one-liner; block *curl* and it pushes to an external git remote. Static rules have no visibility into coverage: nobody can prove a rule set closes every path, or that three individually mundane tools chained together cannot leak. Rule sets therefore converge to one of two failure modes — so tight they break the agent, or so intricate nobody can audit what they permit.

Agent security, then, has two axes that any serious system must satisfy simultaneously: an agent that permits unauthorized flows is **unsafe**, and an agent that refuses valid work is **useless**. Classical information-flow control (IFC) promised the first axis for forty years and repeatedly failed the second — over-blocking benign work, stranding execution after any contact with untrusted data. What changed in 2026 is that APPA made a credible attempt at having both: deterministic flow control with a recovery system attached, evaluated with numbers. One detail from its evaluation is worth flagging early, because it frames the whole field: the authors excluded the popular AgentDojo benchmark because current frontier models natively resist its injection templates — GPT-5.6 Luna showed 0% undefended attack compliance across 160 probed episodes, versus 31% on GPT-4o. The attacks that matter are no longer the crude ones; they are the ones that ride legitimate data flows. That is exactly the terrain APPA chose to fight on.

2 APPA: What It Is and What It Actually Proves

OpenAPPA sits between an agent and its tools and answers one question before every action: *is this data allowed to go to this destination?* It is powered by APPA — the Agentic Permissions Policy Algebra — and is MIT-licensed, vendor-agnostic, and currently published as a "Preview & RFC": settled enough to build against, deliberately open to argument. The formal core is compact enough to state precisely, and precision matters here, because everything this paper later critiques is a boundary condition of these exact definitions.

2.1 The label algebra

> *"A security label is a pair $L = (R, t)$ in the finite product lattice $S = P(U) \times T$, where R is a finite set of authorized reader identities and T is a totally ordered trust chain (e.g., suspicious < trusted)."*
>
> — APPA paper, §3 Policy State

Definition 1 (Security label and lattice). A trajectory carries a label $L = (R, t)$, where $R \subseteq U$ is the set of reader identities authorized to see the trajectory's data and t is a trust level from a totally ordered chain. Labels are ordered by restrictiveness: $(R', t') \leq (R, t)$ iff $R' \subseteq R$ and $t' \leq t$. Combining streams is the component-wise meet $(R, t) \wedge (R', t') = (R \cap R', \min(t, t'))$. Reading a private repo narrows R; reading an unvetted web page lowers t. The paper's Proposition 1 (Collapse) makes composition exact: $\text{fold}(a_1 \cdots a_n)(L_0) = L_0 \wedge a_1 \wedge \cdots \wedge a_n$, and Proposition 2 (Monotone descent and settlement) guarantees labels only ever restrict, stabilizing after finitely many strict descents on the finite lattice.

The threat model is unusually crisp for this literature: the LLM is "strictly policy-untrusted" — it may be adversarially injected, hallucinating, or merely miscalibrated, and may propose arbitrary tool calls — while the trusted computing base (TCB) comprises the reference monitor, deployment profiles and tool contracts, authority resolvers, and data transformations (sanitizers, validators, casts). The model influences which recovery options get tried, "but cannot manufacture authorization." That division of labor is the design's spine, and also — as §3 will show — its softest seam.

2.2 Dual-phase enforcement

Definition 2 (Prospective post-call label). When the agent proposes tool call τ under label L, the monitor computes the post-call label $c_\tau(L) = L \wedge d_\tau$, where d_τ is the tool's declared information contribution, and evaluates release requirements, history predicates over the committed-effect log, and hard gates against this *prospective* label — before dispatch.

Prospective evaluation is not cosmetic. The paper's Proposition 4 shows it is necessary: for composite tools that read and release in one dispatch — its example is *share_legal_packet*, which reads a confidential ledger and emails it externally — a reactive monitor that checks the pre-call label authorizes the egress and notices the ingestion only afterwards. Folding the declared delta in first closes the window before any byte moves.

The second phase closes a different one: after execution, the realized return payload is validated again before admission into model context, because external services can diverge from their contracts. Unannotated returns enter as an Unknown source and flow lazily until a consuming check triggers *cast resolution* — APPA's gradual security typing for rolling out unannotated tools.

2.3 Recovery: the actual innovation

The genuinely new contribution is not the algebra — labels, meets, and monotonicity are Denning's 1976 lattice wearing 2026 clothes — but the recovery layer. A bare "forbidden" makes an agent stall, retry, and fail; APPA instead returns machine-readable remedy plans the policy permits:

- **Sanitizers** transform payloads — masking secrets, redacting PII — so data can flow to a wider audience. Stock ones ship in-box; custom ones, "including model-based ones," plug in with a declared blast radius.
- **Authorities** approve one exact rendered call against a pre-declared mandate — never relaxing the session label. Per the paper, authorities are "implementation-agnostic: deterministic rules, human reviewers, or a secondary model evaluator (an LLM-as-a-judge in auto-approval modes)." Theorem 6 (no unruled inadequate release) and Corollary 7 bound what rulings can do: model tokens cannot forge one, expand a mandate, or confer lingering authority.
- **Subagent forks** isolate an untrusted or confidential read in a disposable child branch seeded with the parent label ($L_{0c} := L_p$), which exits only through abandonment, an ordinary narrowing return, or a schema-attested/sanitized channel — Theorem 8 confines taint to the branch boundary.

The ablation table is the empirical argument that this layer matters: removing fork advertisement drops Bench-Corp completion from 88.0% to 56.5%; removing guided recovery drops it to 35.0% — with zero attacks succeeding in every configuration. The project page rounds this to "37% to 90%"; the paper's own table is the honest version. Either way, the delta is the point: recoverable IFC is what makes deterministic enforcement compatible with an agent that still finishes its work.

2.4 What the benchmarks say — and what they do not

Three tables follow, reproduced from the paper and the project's evaluation page with per-cell values exactly as published. Table 1 is the complete Bench-Corp aggregate (paper Table 2): twenty multi-step enterprise workflows per cell, run under Standard and adversarial "Chaos" profiles, with utility and attack-success rate out of 100 episodes. The permissive arm (APPA-open) is the control that proves the threats are real — strip enforcement and 26–45% of chaos episodes leak. Table 2 is the recovery ablation. Table 3 breaks AgentThreatBench down by OWASP category and exposes where the utility floor lives.

Model · Profile	APPA	APPA-open	F-M	F-N	F-open
GPT-5.6 Luna · Standard	87 / 0	90 / 31	41 / 30	36 / 34	87 / 34
GPT-5.6 Luna · Chaos	89 / 0	91 / 35	36 / 34	38 / 31	92 / 26
DeepSeek V4 Flash · Std	90 / 0	88 / 33	41 / 34	44 / 35	90 / 34
DeepSeek V4 Flash · Chaos	89 / 0	92 / 38	38 / 35	39 / 31	94 / 45
Gemini 3.7 Flash · Std	89 / 0	93 / 30	42 / 31	46 / 27	92 / 31
Gemini 3.7 Flash · Chaos	91 / 0	92 / 30	45 / 26	43 / 29	90 / 29

Table 1 — Bench-Corp complete aggregates (paper Table 2). Cells are utility / attack-success out of 100 episodes per profile. F-M and F-N denote Fides-middleware and Fides-native.
Source: arXiv:2607.24625v2, Table 2. APPA-open enforces no flow policy (control). Fides configurations as evaluated by the APPA authors.

Configuration	Standard	Chaos	Pooled
APPA (guarded, full recovery)	87 / 0	89 / 0	88.0 / 0
appa-nofork (no subagent forks)	57 / 0	56 / 0	56.5 / 0
appa-noremedy (no guided recovery)	35 / 0	35 / 0	35.0 / 0

Table 2 — Bench-Corp recovery-mechanism ablation, GPT-5.6 Luna (paper Table 3). Utility / ASR out of 100 episodes per profile, 200 pooled.
Source: arXiv:2607.24625v2, Table 3. The two ablation arms interact; the paper notes this is not an additive decomposition.

Model · Profile	Memory (of 50)	Autonomy (of 30)	Exfiltration (of 40)
Luna · Standard	28	30	25
Luna · Chaos	29	30	25
DeepSeek · Standard	17	30	30
DeepSeek · Chaos	19	30	34
Gemini · Standard	26	30	35
Gemini · Chaos	27	30	35

Table 3 — Guarded APPA on AgentThreatBench by task category (paper Table 5). Completed episodes out of total; attacks are zero in every cell.
Source: arXiv:2607.24625v2, Table 5. Ten Memory Poisoning, six Autonomy Hijacking, and eight Data Exfiltration upstream tasks (OWASP Top 10 for Agentic Applications).

Three readings fall out of these numbers. First, the abstract's utility floor of 64.2% is not an abstraction: it is DeepSeek V4 Flash on AgentThreatBench Standard — 77 of 120 episodes — and Table 3 shows exactly why. Memory completion runs 17/50 there, because the guarded policy admits no memory writes at the required trust level, so tasks that require writing to memory simply fail closed. The paper is candid about this: "APPA's Memory Poisoning utility trails F-M's with every model." Second, the zero column is uniform — no scored attack succeeded in any of the 1,320 guarded episodes across both suites, models, and profiles. Third, the

overhead is modest and worth quoting precisely, with its provenance labeled: the project's evaluation page (not the paper) reports that on the Tau banking benchmark — 97 tasks, four runs, GPT-5.6 Luna at maximum reasoning effort — guarded APPA completed 151 of 388 simulations at a mean reward of 38.92%, against 156 of 388 and 40.21% for the stock agent, while consuming 1,307,763 mean agent tokens per simulation: +4.22% over stock, with 11,355 policy checks and zero tool-execution errors.

What is actually *proven*, then, is a conditional: given accurate tool contracts, accurate casts and sanitizers, a trusted monitor, and attacks of the classes these suites score, no observed episode leaks, and utility holds between 64.2% and 91%. What is *claimed* on the project page is stronger — "100% resistant to data exfiltration caused by prompt injection or model hallucination," and a comparison table in which Claude Code auto mode scores 90% utility with 10% attack success and Microsoft FIDES 41% with 31%. That comparison deserves a precision flag: the NeurIPS paper evaluates APPA against Fides configurations, but the Claude auto-mode numbers come from the project page and README ("Claude Code auto mode let 10 through across the two suites"), not from the paper's controlled arms. It may well be right; it is not paper-grade evidence, and this paper will not lean on it. The algebra's own slogan — "you cannot prompt-inject an algebra" — survives scrutiny. The question is what the algebra can be told, and by whom. That is §3.

3 The Flaw Taxonomy: Where "Airtight" Leaks

The five flaws below are structural, not incidental: each one follows from a design decision that is also a strength, and each is visible in APPA's own artifacts if the marketing layer is peeled back. The paper's limitations section is unusually honest for the genre — it reads, almost verbatim, like a todo list — and this chapter mostly formalizes what it already concedes. Where we go beyond the paper, we say so.

3.1 F1 — The trusted computing base has probabilistic endpoints

The algebra is deterministic; the system around it is not. Sanitizers may be model-based ("custom ones, including model-based ones, plug in with a clear blast radius," per the project docs). Authorities may be "a secondary model evaluator (an LLM-as-a-judge in auto-approval modes)" — the paper's own words. Annotators make model calls, metered as external-call time in the observability spec. Every one of these is an LLM endpoint inside the TCB, and every one of them reads raw, untrusted, attacker-influenceable data in order to label it. The safety theorems hold *given correct labels*; the endpoints that produce labels are the same class of artifact as the thing being guarded. An attacker who cannot prompt-inject the algebra can still try to prompt-inject the annotator that feeds it.

There is an irony worth naming, because APPA's marketing uses it as a weapon against auto-modes: the site criticizes judge-based systems because "harnesses hide tool outputs from them, so the judge never sees the data at all." True — and the flip side is that any labeler accurate enough to matter must *see* the data, which makes it an injection surface positioned one step upstream of the decision. The attack class this opens is concrete: homoglyph PII that a sanitizer's regex-equivalent passes untouched; a secret split across two innocuous fields whose halves recombine downstream; zero-width watermarks that survive redaction. The

algebra transports all of it faithfully. The guarantee is label ⊆ enforcement, not label = reality.

3.2 F2 — The semantic gap

"Is this string a person's name?" is a fact about the world, not about syntax. A finite lattice of audiences and trust levels is a lossy compression of a world whose privacy-relevant properties are unbounded — and the attacker lives exactly in the compression loss. This is not a bug APPA can patch; it is the permanent condition of every label-based IFC system since Denning. What APPA adds is that its labels are sometimes assigned by corpus-trained models (F1), which makes the compression loss *adversarially steerable* rather than merely random. The right engineering response is not to demand perfect labels — impossible — but to (a) ground what can be grounded cryptographically, so that at least channel-provenance facts are unforgable, and (b) detect leakage semantics-first at the decision boundary, which is precisely what Chapter 6's adjudicator does.

3.3 F3 — The monitor is not provably total

> *"Covert channels from a malicious model remain out of scope, and branching isolates prompt context rather than rolling back committed external effects."*
>
> — APPA paper, §8 Limitations (reference markers omitted)

The reference monitor sits "at tool dispatch and protocol gateways (e.g., MCP)." Two classes of motion escape that perimeter. First, raw egress: an agent with a shell tool can open a socket that never passes a tool contract; unless the sandbox's network policy is compiled in lockstep with the algebra, enforcement and reality diverge exactly where exfiltration is easiest. Second — and unfixable in principle — covert channels: an agent that has read a secret and then merely *chooses* which public tool to call next encodes one bit per decision, observable in commit logs, API timing, or choice of emoji in a public post. Lampson proved in 1973 that confinement cannot close these; the paper correctly scopes them out, and the "zero observed attacks" claim should be read as "zero attacks of the classes scored." There is also a third, quieter gap the paper concedes: effects enter the log as reported, so a tool that succeeds externally while reporting indeterminate simply vanishes from history — exactly-once guarantees "hold against the world only with outcome reconciliation or idempotency keys at the tool boundary." A lying tool is outside the algebra's jurisdiction.

3.4 F4 — The monotone ratchet is an operational time bomb

> *"Five repetitions reuse 20 or 24 task designs and are not independent tasks. Coverage is limited to three models and one policy per defended arm; finite zero-ASR observations do not prove attacks impossible, and APPA's Memory Poisoning utility trails F-M's with every model."*
>
> — APPA paper, §8 Limitations

Monotonicity is theoretically elegant and operationally harsh. Labels only restrict; an authority approval licenses exactly one call and relaxes nothing; there is no declassification lifecycle — no "publishable after 30

days," no "k-anonymized aggregates are public," no retention decay. The visible cost is already in Table 3: memory tasks fail closed at 17–29 of 50 because writing to memory requires a trust level the ratcheted session no longer holds. The invisible cost is behavioral: operators confronting a session that can no longer do anything useful have exactly one sanctioned move — restart it, discarding context — and an unsanctioned one: loosen the policy. Enough operators choosing the second erodes the security story from the inside. IFC history says this is not speculation; it is the standard death spiral of deployed taint tracking. Two published numbers make the fatigue dimension real: Bench-Corp tasks are bounded workflows (twenty designs, reused across five repetitions — the paper says so), and nothing in the evaluation measures approval rates, time-to-approve, or fatigue curves over hundred-turn sessions. The ratchet has never been stress-tested at the horizons where production agents actually live.

3.5 F5 — Benchmark epistemics

Bench-Corp — the suite supplying the 88–90%-versus-36–46% humiliation of Fides — was authored by the APPA team, and the project's own evaluation page acknowledges it "enforced rules those [Fides] configurations did not support, including recipient authorization, out-of-band approval, and required action ordering." Scoring a system on rules its competitor does not implement measures the rule set, not the engines. The paper flags this as "configuration-specific rather than an exhaustive baseline comparison," and also concedes the floor effects that got AgentDojo excluded. None of this is fraud; all of it is context that headline tables omit. The correct reading of Table 1 is not "APPA beats Fides" but "under one team's policy grammar, deterministic flow control with recovery dominates judge-based control on both axes." Independent replication does not yet exist. Until it does, the strongest defensible claim is the paper's own: zero observed attacks, finite sample, three models, one policy per defended arm.

> **The paper's own closing caveat**
> "Misconfigured contracts, authorities, or sanitizers can create false assurance. Deployments should audit rulings, minimize authority mandates, and use APPA as one defense-in-depth layer." — §8, Broader impact. This paper's position is simply that the caveat is the architecture: build the other layers explicitly.

4 The Five Walls: What Forty Years of Theory Actually Proves

Before proposing anything, it is worth being precise about why "foolproof agent security" is not an engineering goal but a category error. Five results — two theorems, two principles, one empirical constant — stand between ambition and airtight. Each wall below is load-bearing; removing any one of them is a result worthy of a Turing Award, and several people have received one for trying.

1. **Wall 1 — Rice's theorem (1953).** "Does this action leak?" is a non-trivial semantic property of an open-ended program (the agent, the tools, the world). Deciding it in general is equivalent to deciding the halting problem. Perfect *static* enforcement is therefore impossible; every practical system — APPA

included — enforces a decidable syntactic approximation of an undecidable semantic property, and the approximation gap is exactly where attacks live.

2. **Wall 2 — Lampson's confinement result (1973).** A process that observes a secret and controls anything publicly observable has a covert channel; capacity can be throttled but never zeroed without removing all observable choice. For agents: tool choice, timing, and even error messages are observable. No content filter closes this; it is information-theoretic.
3. **Wall 3 — The semantic gap.** Labels are a finite compression of world semantics (F2). No finite label system assigns ground truth to "is this sensitive" across an unbounded input space.
4. **Wall 4 — TCB conservation.** Trust does not vanish under scrutiny; it relocates. Minimize it, distribute it, fracture it across hostile implementors — but every verifier needs verifying, all the way down. The demand for a trust-free security system is a demand for a first unproven mover.
5. **Wall 5 — The human hole.** The last authority in every deployed system is a person, and the person approves phishing emails at measurable rates. Human-in-the-loop is a blast radius, not a resolution.

Aviation is the canonical existence proof of what to do with five walls: commercial flight is not crash-proof — it is a system in which crashes are rare, survivable, attributed, and engineered out of existence iteratively. That outcome was built from redundancy, black boxes, an independent accident investigator, and an insurance market that prices risk — not from a taller fence. The translation to agent security is nearly mechanical, and it dictates PANOPTES' shape: multiple independent enforcers (O7), tamper-evident action logs (O7), synthesized canaries and attribution (O4), escrowed time making effects survivable (O6), and risk that is priced and insurable rather than denied (O5).

This is also the honest way to read APPA's strongest number. "Zero observed attacks in 1,320 guarded episodes" is a real and impressive result — and the paper itself appends the correct epistemic tag: "finite zero-ASR observations do not prove attacks impossible." The claim that survives all five walls is not *zero leaks*. It is *a known leak rate — attributable, reversible, unprofitable, and shrinking asymptotically*. The rest of this paper is an architecture for exactly that property, and a redefinition the industry should adopt: **airtight means the asymptote, not the wall.**

5 PANOPTES: An Architecture for Survivable Agent Security

PANOPTES is not a better algebra; it is the city built around the cathedral. Its fast path is an APPA-class label engine — everything Chapter 2 described, contributing its deterministic microsecond-scale verdict on every call. Its contribution is everything the algebra cannot, even in principle, see: the causal behavior of the decision function, the economics of the adversary, the reversibility of the world, and the distributed trust that keeps any single monitor from becoming the crown jewel. Nine organs, each specified with the wall or flaw it answers and a build-complexity grade (L1: ships in weeks with existing libraries; L2: quarters; L3: a small team-year; L4: research program).

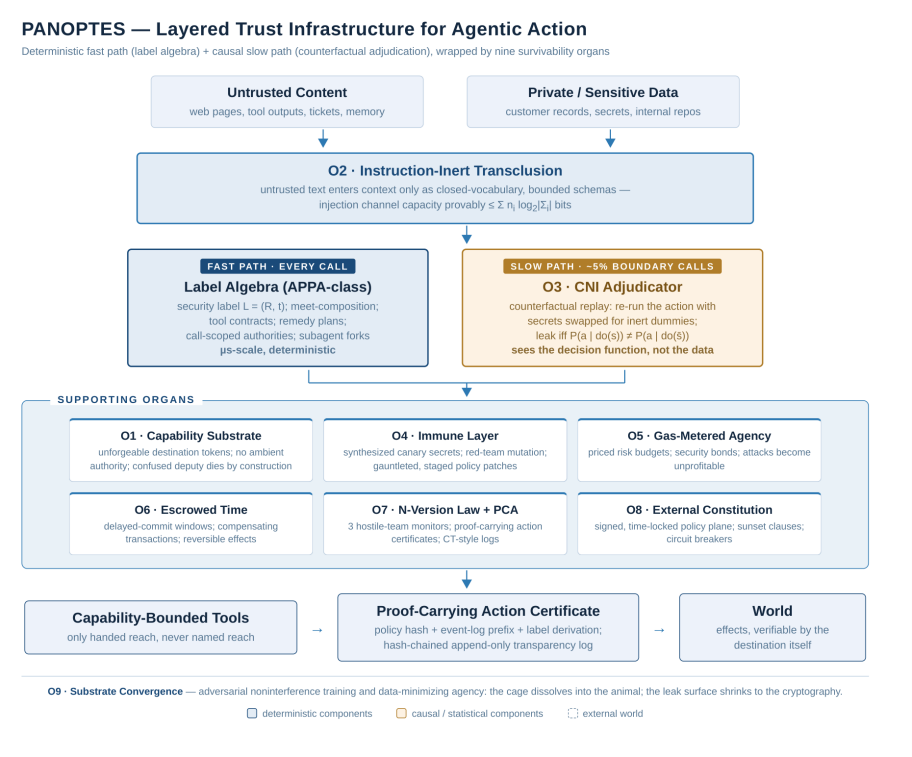


Figure 1 — PANOPTES overview: deterministic fast path plus causal slow path, wrapped by the nine survivability organs.

5.1 O1 — Capability substrate: hands, not words

The single most airtight move available is to police anatomy rather than intent. In an object-capability system, the agent never *names* destinations at all: every reachable destination is an unforgeable token the agent was handed — by the user, the policy, or another capability it already holds. There is no ambient authority to abuse, so the confused-deputy problem dies by construction: the deputy can only use what it was given, and possession is revocable. This addresses F3's raw-egress crack directly — sockets, files, and remotes become capabilities the policy grants or withholds, which means the sandbox and the algebra stop being parallel, separately-bypassable systems. The APPA authors would recognize the lineage: their contracts describe what tools *do*; capabilities describe what tools *can touch*, and the two compose. Build: L2 — capability discipline for MCP-style tool graphs is a well-understood refactor (answer to F3; complements Walls 1–2 by shrinking the enforcement surface to what is reachable at all).

5.2 O2 — Instruction-inert transclusion: schemas as narrowband

> **Definition 3 (Injection channel capacity).** Let untrusted content be admitted to model context only through a closed schema with fields $f_1, \ldots, f_k$, each over a bounded vocabulary Σ_i with length cap n_i, and with no field permitted to interpolate into instructions. The channel capacity available to an attacker is bounded: $C \le \sum_i n_i \log_2 |\Sigma_i|$ bits — finite, computable, and tunable by vocabulary design.

The root cause of prompt injection is architectural: the context window is one undifferentiated namespace where instructions and data share an address space. APPA quarantines dangerous prose in subagent branches and projects tool outputs through bounded schemas — its Autonomy projection "categorizes every non-spam record as normal," so untrusted content cannot promote itself to *important*. PANOPTES generalizes this into an ingress law: untrusted text may enter the primary context only as typed, bounded, closed-vocabulary projections with no imperative fields and no string interpolation into prompts. Free prose is a Turing-complete attack channel; a closed schema is a channel with a provable bit-rate. You cannot stop injection — you can put an upper bound on its bandwidth, meter it, and alarm when traffic approaches it. Nobody in the agent-security literature is publishing injection-channel capacity numbers yet; that measurement alone is a paper. Build: L2 (answers F1/F2 at ingress; shrinks Wall 3 to a bounded-rate adversary).

5.3 O3 — Counterfactual noninterference adjudication

The crown jewel; it gets Chapter 6 to itself. In one sentence: define leakage causally, as a property of the model's decision function, and test it by replaying boundary actions with the secrets swapped for inert dummies. The algebra sees bytes; the adjudicator sees behavior. Build: L3 (answers F1, F2, and Wall 2's observable-choice channel at the decision boundary).

5.4 O4 — The immune layer

Static defenses decay; immune systems do not, because they generate attacks faster than adversaries do. Continuously synthesize canary secrets — fake API keys, fake customer records, uniquely numbered and registered — and seed them through the live system as ordinary traffic. Any canary surfacing anywhere it should not (a public repo, an outgoing email, a web request) is a detected breach with a forensic trail. A red-team agent mutates injection payloads against the deployed policy like a pathogen seeking antibody gaps; every successful variant becomes a candidate patch. Patches run a gauntlet before rollout: a formal dead-end check (did the policy actually block the variant, or did the attack simply fail?), a canary replay, a regression suite, then staged deployment with automatic rollback. This is affinity maturation for policy, and it converts "we blocked 100% of the attacks we thought of" into "we manufacture attacks continuously and metabolize the survivors." It also directly operationalizes the paper's own broader-impact advice — audit rulings — by automating the audit. Build: L2–L3 (answers F5's static-benchmark epistemics with a generative one).

5.5 O5 — Gas-metered agency: make attack a bad business

Binary allow/deny at the margin optimizes the wrong thing: a fenced agent slam-into-walls until it stalls or a human relents. Instead, price every action path by (label delta × irreversibility × blast radius): dangerous-but-legal paths become *expensive*, remedy paths become *cheap*, and the agent plans around prices the way it already plans around token costs. Layer slashing on top: agents (or their operators) post security bonds to operate at elevated capability, and verified misbehavior burns stake. The adversary's ledger now reads p(success) → 0 under the algebra, × p(detection) → 1 under the canaries, − stake, − penalties. Mechanism design closes what enforcement cannot: you cannot make attack impossible, but you can make it unprofitable, which is the actually achievable victory condition. Build: L2 (answers Wall 5 economics; complements O4).

5.6 O6 — Escrowed time: the undo organ

Irreversibility is the real enemy — a blocked leak is an incident, a completed one is a disclosure. So irreversible effects acquire a delayed-commit window: the email sits signed-but-unsent for N seconds, challengeable by the adjudicator (O3) or a circuit breaker; writes to shared systems land as compensating transactions; world-facing actions snapshot prior state with a notarized rollback plan. The paper's limitations concede the exact gap this closes: branching "isolates prompt context rather than rolling back committed external effects." Time is the last line of defense and the only one money cannot argue with. Build: L2 per integration (answers F3's committed-effects gap; makes the blast radius of Walls 1–5 survivable).

5.7 O7 — N-version law and proof-carrying actions

TCB conservation says trust relocates; diversity says you can fracture it. Run the policy monitor as an n-version system — three independent implementations by three mutually hostile teams, majority rule — and compromise now requires subverting a majority of heterogeneous codebases and their witnesses at once (Avizienis's 1985 prescription, applied to policy). Every permitted effect then ships with a proof-carrying action certificate: the policy hash, the event-log prefix, the label derivation that authorized it — verifiable *by the destination itself*, independent of the monitor that issued it. Certificates chain into an append-only, hash-chained, gossip-replicated action log: Certificate Transparency for agency. This kills three birds the paper names: the monitor stops being a crown jewel; third parties can audit behavior claims without trusting the operator; and a lying tool's divergence between reported and realized outcomes becomes detectable at reconciliation. Build: L3 (answers Wall 4 and F5).

5.8 O8 — The constitution outside the cage

Policies live in a plane the agent structurally cannot reach: signed, quorum-gated, time-locked, and — critically — *expireable*. Authority grants carry sunset clauses, so approval fatigue and privilege hygiene become the same mechanism: everything lapses unless affirmatively renewed. Org-wide circuit breakers give an operator a single, tested, physical action that halts all agents on evidence of compromise — the big red book of aviation, not a Slack thread. Gödel's lesson, applied as architecture: any system smart enough to negotiate with its own rules is smart enough to subvert them, so the law must be structurally unreachable by the governed. This matters double for the self-modifying agents already on roadmaps. Build: L2 (answers Wall 5; hardens O1–O7 against the agent itself).

5.9 O9 — Substrate convergence: the endgame

The endgame is that the cage dissolves into the animal. Two convergent programs: **adversarial noninterference training** — perturb untrusted-context tokens during training and penalize any change in action distribution, making injection-resistance a property of the weights rather than a runtime fence; and **data-minimizing agency** — agents that reason over zero-knowledge predicates and encrypted capabilities ("I know this record matches the query; I never see the record"), shrinking the leak surface to the cryptography itself. Neither is deployable this year; both are research programs worth staking claims in now, because whichever lab ships substrate-level noninterference makes every runtime organ cheaper. Build: L4 (answers everything, eventually).

Organ	Mechanism	Answers	Build
O1 Capability substrate	Unforgeable destination tokens; no ambient authority	F3, confused deputy	L2
O2 Inert transclusion	Closed schemas; provable channel capacity bound	F1, F2 at ingress	L2
O3 CNI adjudication	Counterfactual replay; causal leakage test	F1, F2, Wall 2 residue	L3
O4 Immune layer	Synthesized canaries; red-team mutation; patch gauntlet	F5, detection	L2–L3
O5 Gas-metered agency	Priced risk budgets; bonds and slashing	Wall 5 economics	L2
O6 Escrowed time	Delayed commits; compensating transactions	F3 committed effects	L2
O7 N-version + PCA	3 hostile-team monitors; proof-carrying certificates; CT logs	Wall 4, F5	L3
O8 External constitution	Signed, time-locked, sunsetting policy plane; breakers	Wall 5, self-modification	L2
O9 Substrate convergence	Noninterference training; data-minimizing agency	All, asymptotically	L4

Table 4 — The nine organs, their targets, and build complexity.
Complexity: L1 = weeks; L2 = quarters; L3 = team-year; L4 = research program.

6 The Crown Jewel: Counterfactual Noninterference Adjudication

Everything in this paper funnels here, because CNI is the one proposal that addresses the residue class rather than its shadow. The setup: Goguen and Meseguer defined noninterference counterfactually in 1982 — high inputs do not leak if low outputs would have been the same without them. Every IFC system since, APPA included, enforces a tractable *syntactic approximation* of that semantic property: track labels, forbid label-violating flows. The approximation is sound (no tracked flow violates noninterference) but incomplete — and incompleteness is not a footnote when the adversary chooses where to live.

6.1 Leakage as a causal property of the decision

> **Definition 4 (Causal leakage).** Let A be the agent's next-action distribution conditioned on its current context, and let s be a high-label input in that context with s′ an inert dummy of equal type and format. Action a leaks s iff $P(A = a \mid do(s)) \neq P(A = a \mid do(s'))$ — the interventional substitution of the secret changes what the agent does. Equivalently: a is leak-free with respect to s at tolerance ε when the divergence $D_{KL}(P(A \mid do(s)) \parallel P(A \mid do(s'))) \leq \varepsilon$.

The formulation is Pearl's do-calculus pointed at a model. The subtle point that makes it worth a paper: leakage so defined is a property of the *decision function*, not of the data. A secret that the model ignores causally cannot leak through the action, however badly it is labeled; a secret the model operationalizes leaks through the *choice among public tools*, however perfectly it is labeled. This is exactly the blind spot shared by every taint tracker: the covert channel — one bit per public decision — and the semantic leak that passes redaction (the model has seen enough to act on what it cannot quote) are both properties of behavior under intervention, invisible to byte-level labels.

> **Proposition P1 (Residue class; proof sketch).** There exist trajectories that satisfy APPA label discipline at every step yet transmit secret information to a public observer. Construction: a trajectory reads a high-label record, then faces two policy-identical public actions (post issue titled "A" or "B"). The label algebra sees no flow — both actions are public, no field carries secret bytes. The agent conditioned on the secret chooses deterministically between them, encoding one bit per decision, log n bits per session for n decisions. Under Definition 4, $D_{KL}(P(A \mid do(s)) \parallel P(A \mid do(s'))) > 0$ — observed, and attributable. No content filter sees this; counterfactual replay does. (Lampson 1973 supplies the general impossibility; CNI supplies detection and a bound.)

6.2 Runtime: counterfactual replay on the slow path

The runtime shape is a two-tier monitor. The fast path is the APPA-class algebra, sub-millisecond, on every call — its verdicts unchanged. The slow path triggers only on *boundary actions*: public-bound proposals that follow sensitive reads, which the label engine itself identifies — in production traffic this is a small fraction of calls (APPA's own numbers suggest single-digit percentages of calls carry sensitive-read history; the exact rate is deployment-specific and should be measured, not assumed). For a flagged action, the adjudicator re-runs the decision k times under intervention: high-label inputs replaced by inert dummies matched in type, length, and format. If the action distribution is insensitive to the substitution — the agent files the same issue, with the same content, whether or not the secret was there — the action is *observed noninterferent at the point of decision*, and dispatches. If the distributions diverge beyond ε, the action depends on the secret in a way its label path never showed: block, and return a remedy plan (sanitize; ask authority; fork) exactly as APPA already does.

Two engineering caveats keep this honest. First, counterfactual sensitivity conflates attack with *legitimate dependence* — the whole point of many workflows is that the output depends on the private input (a draft email summarizing the ticket *should* vary with the ticket). The adjudicator therefore tests dependence at the *public-boundary* only, under the destination contract: the question is not "does the action depend on the secret" but "does it depend on it in ways its destination is not authorized to receive," operationalized by comparing against dummies carrying the destination-authorized projection. False positives are the design's main risk and are controlled by ε calibration, dummy quality, and per-workflow baselines — the same trade APPA's sanitizers already make, moved from bytes to behavior. Second, cost: k re-derivations per boundary call multiplies inference for that call by (k+1)/1. With k = 3 and 5% of calls flagged, expected overhead is roughly 15% of decision tokens — compare APPA's measured +4.22% total. That is the price of seeing behavior; it is also a tunable dial, not a constant.

6.3 The benchmark protocol

The evaluation writes itself, which is the mark of a good attack on a real gap. Assemble paired episodes from AgentThreatBench's exfiltration and memory-poisoning tasks plus Bench-Corp's workflows; tag every high-label value with a registered canary; run each episode twice — once with the true secret, once with the inert dummy — under identical seeds; score ASR (canary observably reaches an unauthorized destination), utility (task completion), and overhead (tokens and latency). Report the residue-rate: the fraction of episodes where the algebra passes and the adjudicator diverges. That number is the whole thesis made empirical — if it is zero, CNI is dead and APPA is more airtight than this paper fears; if it is non-zero, the field has been measuring the shadow instead of the threat. Either outcome is publishable; only one of them is comfortable.

7 Roadmap: Contribute to APPA or Build PANOPTES?

"Build a better system" is the losing move, and this paper should be explicit about why: a rival policy algebra competes against a NeurIPS-accepted paper, an RFC community, and Archestra's distribution — head-on, with none of the leverage. The market, however, is not policy algebras; it is *trust infrastructure for agentic action*, and its unsupplied layers are exactly Chapters 5–6. The correct strategy is sequenced contribution: APPA as fast path and RFC collaborator first, adjudicator second, stack third. "Contribute or compete" is a false binary; the sequencing *is* the answer.

7.1 Phase 1 (months 0–3): feed the RFC while it is soft

APPA is published as a Preview & RFC, "deliberately open to argument" — maximum design leverage exists right now, while the config and wire surfaces are still allowed to break. Two contributions fit the RFC's own stated concerns and require no new theory. **Provenance-channel labels**: base label assignment on cryptographic facts about authenticated channels — signed tool responses, mTLS identities, attested origins — and constrain model-based annotators to *raising* labels, never lowering them. This converts F1's softest seam from "LLM inside the TCB" to "fail-closed refiner over cryptographic ground truth," a strict hardening the authors would have difficulty rejecting on the merits. **Policy model-checking**: compile appa.toml plus the tool graph into a TLA+/Alloy model, enumerate reachable label states exhaustively, and emit counterexample traces — "this policy dead-ends after read X then read Y; no remedy exists for the tool the task requires." Their *appa replay* is testing; this is verification, and it directly attacks F4 (the ratchet's dead ends become build failures instead of production incidents) using the declarative TOML they already chose for exactly this reason. Success criteria: both landed as RFC items with working prototype checks; working familiarity with the codebase as a side effect.

7.2 Phase 2 (months 3–12): the CNI prototype and the killer demo

Build the adjudicator around APPA as its fast path — the two-tier monitor of §6.2, instrumented on the AgentThreatBench harness with canary-tagged secrets. The milestone that matters is singular: **an attack that passes the algebra and is caught by the adjudicator** — a tool-choice covert channel or a redaction-passing semantic leak, reproduced on their own benchmark, with the paired-episode protocol of §6.3. If it exists, it is simultaneously the NeurIPS submission and the launch argument, and the honest possibility that it does not exist is what makes the experiment science rather than marketing. Success criteria: residue-rate measured (non-zero result published, or the negative result published — both are contributions); overhead within the §6.2 envelope; remedy-plan integration so flagged actions degrade gracefully rather than failing flat.

7.3 Phase 3 (year 1+): the trust layer

With the adjudicator proven, assemble the vendor-neutral stack: inert transclusion (O2) as an ingress library, proof-carrying action certificates and transparency logs (O7) as infrastructure every agent vendor needs and nobody ships, escrowed-time adapters (O6) for the two or three effect surfaces that matter per vertical, and the immune layer (O4) as the operational service that keeps all of it current. This is the layer airlines and banks can underwrite — which is the point, because insurability is what converts security from a cost center into a market.

Phase	Deliverable	Depends on	Primary risk	Success criterion
1 · RFC contributions	Provenance labels; TLA+/Alloy policy model-checker	APPA RFC acceptance; TOML schema	RFC churn breaks surface	Both merged; prototype checks public
2 · CNI prototype	Two-tier monitor on AgentThreatBench harness	Phase 1 codebase familiarity	Residue-rate measures zero	Published residue result; overhead ≤ ~15% decision tokens
3 · Trust layer	O2 + O7 + O6 + O4 as services	Phase 2 evidence; adopters	Vendor NIH; standardization drag	First external audit of an action log completed

Table 5 — Phased plan with success criteria and risks.
Negative results in Phase 2 are deliverables, not failures: a measured zero closes the question this paper opens.

8 The Residual Risk Budget: An Honest Closing Account

R = C·E·T·H
residual risk = covert capacity × label error × TCB compromise × human error

each term
minimized, measured, priced, insured — none zero, all converging

Every system this paper has discussed — APPA, Fides, auto-modes, PANOPTES at full build-out — leaves a residual. The engineering sin is not the residual; it is leaving it *unaccounted*. PANOPTES therefore closes with the account: covert-channel capacity C (bounded by O2's ingress caps and metered by O3), label error rate E (reduced by provenance grounding, bounded by sanitizer certification and measured by the immune layer), TCB-compromise probability T (fractured across n-version monitors and auditable through proof-carrying logs), and human error H (priced, bonded, and sunset by O5 and O8). Multiplication is the honest operator: no term is zero, every term has an owner and a metric, and the system's public claim is the product and its trajectory — not a boast of absence.

That is the redefinition this paper asks the field to adopt, and it is more demanding than the one it replaces. "Airtight" as "zero leaks" is marketing that forty years of results — Rice, Lampson, and the semantic gap —

have priced at infinity. "Airtight" as "a known leak rate, attributable, reversible, unprofitable, and shrinking asymptotically" is an engineering commitment you can be held to: the asymptote, not the wall. APPA, read carefully, already points at this — its own limitations section is practically a contributions wishlist, and its recovery machinery is the first production-grade answer to the utility axis. The right relationship between the two systems named in this paper's title is the one aviation found between the airframe and the air-traffic system: the algebra is the fast, local, deterministic control loop; PANOPTES is everything that makes the whole sky survivable — the redundancy, the black box, the immune system, the insurance, and the accident investigator. Do not beat the cathedral. Become the city around it.

Engineering Annex — The Reference Implementation

Since the architecture above was drafted, its deterministic fast path has been built, audited, and stress-tested as a working reference implementation (panoptes-core, MIT). This annex records the measured state so the paper can be read against evidence rather than intention. Scope discipline: the implementation covers the label lattice (Chapters 2–3), the cross-session memory ledger, canary interdiction, the injection screen, inert transduction, and the fail-closed ratchet — the organs whose guarantees are checkable deterministically. CNI, escrowed authority, and n-version law remain open Phase-2/3 work exactly as the roadmap states; they are simulated pedagogically, not shipped.

Claim	Measured value	How verified
Corpus accuracy, organs on	24/24 attacks blocked, 16/16 benign passed	40-scenario deterministic replay against locked goldens (panoptes bench)
Real-binary blind spots	OpenAPPA v0.30.0: 16/24 attacks, both structural taxa missed	Head-to-head on the same corpus, replay mode
Decision latency	0.62 µs mean, 3.8 µs p99 in-process (1.63 µs mean over the mixed soak)	Hot-loop micro-benchmark, per-arm stated
Sustained throughput	615,270 decisions/s over a 20M-decision soak	Block-timed mixed load, six organ hot paths interleaved
Soak integrity	Determinism byte-identical pre/post; heap slope 1.1 MB/M (bounded distinct-content footprint)	512-decision signature replay; RSS checkpoints each million
Session churn	100K fresh sessions, zero verdict-pattern mismatches	Construct → enroll → poison → 4-decision pattern check
HTTP sidecar	164,000 requests, 0 errors; auth gate 1,000/1,000 rejected	Concurrency ladder 1→128 against the shipped serve mode
Red-team at scale	0/1M label-gated evasions; 0/500K benign false positives	3.5M seeded composite-mutation probes (13 types, 1–4 layers)
Regression suite	109 tests, 54,640 assertions, 0 fail	bun test, includes audit hardening regressions

Table 6 — Reference implementation record. All figures are this project's own measured runs on a 2-vCPU host (Bun 1.3.14); they are engineering evidence for the fast path, not evaluations of OpenAPPA.
Known, measured gaps are retained deliberately: transformed-payload canary evasions (base64/hex/rot13) and frozen-screen marker evasions are reported, not hidden — they are the documented reason the canary scan is a tripwire while the label lattice is the wall. Version 1.2.1 adds zero-padded 96-bit key segments, unified egress sink parameters, and a capped MCP session table over the audited 1.2.0 build; goldens unchanged.

What this record does and does not support: it demonstrates that the deterministic core of PANOPTES is buildable, fast enough for the agent hot path, stable under adversarial load, and honest about its content-side limits. It does not evaluate CNI (the centerpiece of Chapter 6), does not measure utility on the APPA corpora with recovery machinery, and contains no field deployments — the residue-rate experiment of §6.3 remains the decisive open question, now with an engine ready to run it.

References

Attribution discipline used throughout: results labeled "paper" derive from arXiv:2607.24625v2 and its tables; results labeled "project page," "README," or "evaluation page" derive from openappa.com and the GitHub repository and are not part of the NeurIPS paper's controlled arms; external results (Codex auto-review figures) are relayed as reported by secondary sources. Quotes from the paper's §8 reproduce its text with reference markers omitted.

Kravchenko, A., Liventsev, V., Konstantinov, I., Iskhakov, I., and Kukuy, M. APPA: Recoverable Information-Flow Control for Real-World LLM Agents. arXiv:2607.24625v2 [cs.CR], revised Aug 2026. Accepted to the NeurIPS 2026 Workshop on Agents in the Wild.

OpenAPPA project documentation: "What is OpenAPPA," "How it works," "Evaluation," "Comparison vs Cedar." openappa.com, accessed October 2026.

archestra-ai/OpenAPPA. GitHub repository, MIT license, README. github.com/archestra-ai/OpenAPPA, accessed October 2026.

Costa, M., Köpf, B., Kolluri, A., Paverd, A., Russinovich, M., Salem, A., Tople, S., Wutschitz, L., and Zanella-Béguelin, S. Securing AI Agents with Information-Flow Control. arXiv:2505.23643, 2025. (As cited in [1], reference [4].)

UK Government BEIS / Inspect Evals. AgentThreatBench: OWASP Top 10 for Agentic Applications. github.com/UKGovernmentBEIS/inspect_evals.

Anthropic. Model Context Protocol (MCP) Specification. modelcontextprotocol.io, 2024. (As cited in [1], reference [2].)

Denning, D. E. A Lattice Model of Secure Information Flow. Communications of the ACM 19(5), 1976.

Goguen, J. A. and Meseguer, J. Security Policies and Security Models. IEEE Symposium on Security and Privacy, 1982.

Lampson, B. W. A Note on the Confinement Problem. Communications of the ACM 16(10), 1973.

Rice, H. G. Classes of Recursively Enumerable Sets and Their Decision Problems. Transactions of the AMS 74, 1953.

Myers, A. C. and Liskov, B. Protecting Privacy Using the Decentralized Label Model. ACM Transactions on Software Engineering and Methodology 9(4), 2000.

Pearl, J. Causality: Models, Reasoning, and Inference. Cambridge University Press, 2nd ed., 2009.

Dwork, C., McSherry, F., Nissim, K., and Smith, A. Calibrating Noise to Sensitivity in Private Data Analysis. Theory of Cryptography Conference, 2006.

Avizienis, A. The N-Version Approach to Fault-Tolerant Software. IEEE Transactions on Software Engineering 12(12), 1985.

Laurie, B., Langley, A., and Kasper, E. Certificate Transparency. RFC 6962, IETF, 2013.

Bytes #525: "Childproofing the ungovernable." Fireship / bytes.dev, September 29, 2026. (Secondary source for vendor-reported Codex auto-review figures.)